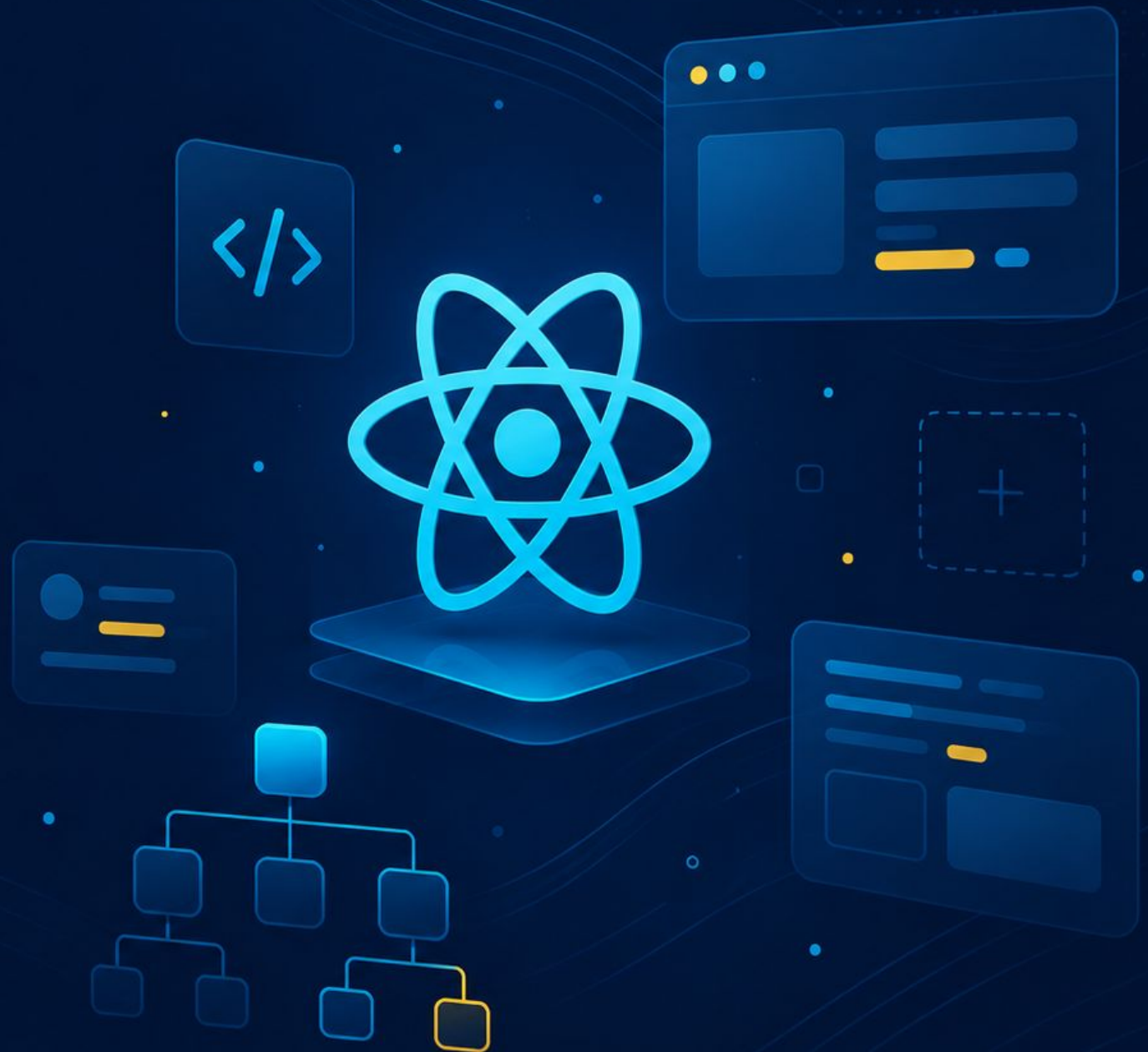


INTRODUCCIÓN A *React*



ISW-521 · Programación en Ambiente Web I — UTN San Carlos · 2026-II

Joseph Moya · Brandon Prado · Integrante 3

Lo que vamos a ver

1

Fundamentos

El problema, HTML/DOM y el enfoque declarativo

Joseph Moya

2

Cómo se usa React

Librería, ReactDOM, JSX y componentes

Brandon Prado

3

Datos e interacción

Props, eventos y estado + demo

Juan Diego Matute

¿Qué problema resuelve React?

Construir interfaces que cambian con la interacción del usuario, sin volverse un caos a medida que la app crece.

Interfaces interactivas

Botones, listas, formularios... cosas que responden y cambian en pantalla.

UI siempre en sync

Cuando los datos cambian, la interfaz debe reflejarlo sin errores.

Código mantenible

Piezas reutilizables en vez de instrucciones manuales repetidas.

HTML → DOM → UI

HTML

El archivo que el servidor envía. Es el contenido inicial de la página.

DOM

El navegador convierte el HTML en un árbol de objetos (padres e hijos). Es el contenido actual, y puede cambiar.

UI

Lo que el usuario ve y con lo que interactúa en pantalla.

Clave: el HTML es el contenido **inicial**; el DOM es el contenido **actualizado** (lo que cambia con JavaScript). Manipular el DOM permite agregar, cambiar y borrar elementos.

Imperativo vs. Declarativo

Imperativo (JS a mano)

```
const app = document
  .getElementById('app')
const h = document
  .createElement('h1')
h.textContent = 'Hola'
app.appendChild(h)
// el CÓMO, paso a paso
```

Declarativo (React)

```
root.render(
  <h1>Hola</h1>
)

// el QUÉ querés mostrar.
// React calcula el cómo
```

 **Imperativo** = darle al chef el paso a paso para hacer la pizza. **Declarativo** = pedir la pizza y no preocuparte por cómo se hace.

¿Qué es React?

Una librería de JavaScript para construir interfaces de usuario interactivas.

Librería

Te da funciones (APIs) para construir UI, pero vos decidís dónde usarlas. No te impone estructura.

Le decís a React qué querés en pantalla

...y React se encarga de cómo actualizar el DOM por vos.

React Declarativo : Nosotros decimos que queremos mostrar y React se encarga de actualizar la pantalla.



¿Y Next.js?

Es un framework construido sobre React: le agrega rutas, manejo de datos y optimizaciones.

ReactDOM: createRoot y render

```
const domNode = document
  .getElementById('root')

const root = ReactDOM.createRoot(domNode)
root.render(<h1>Hola React</h1>)
```

createRoot(nodo)

Elige un elemento del DOM donde vivirá tu app de React.

root.render(ui)

Dibuja tu UI de React dentro de ese nodo.

En un proyecto real (Vite), esto vive en main.jsx y se importa con `import { createRoot } from 'react-dom/client'`.

JSX: HTML dentro de JavaScript

Parecido a HTML, pero es JavaScript

```
const nombre = 'React'

root.render(
  <h1>Hola {nombre}</h1>
)
```

Los navegadores no entienden JSX.

Un compilador (Babel) lo traduce a JavaScript normal.

Las llaves { } = "modo JavaScript".

Dentro de las llaves podés poner cualquier variable o expresión de JS.

Componentes: piezas de LEGO

```
function Header() {  
  return <h1>Mi título</h1>  
}  
  
// se usa como una etiqueta:  
root.render(<Header />)
```

Son funciones

que retornan UI (JSX).

PascalCase

se escriben con mayúscula: Header, no header.

Se anidan

unos dentro de otros → forman un árbol de componentes.

Componentes = piezas reutilizables. Si necesitás cambiar una parte de la UI, tocás solo ese componente.

Cada componente es una función que devuelve JSX.

Props: del padre al hijo

```
function Card({ title }) {  
  return <h2>{title}</h2>  
}  
  
<Card title="React" />
```

Como los atributos de HTML

(src de img, href de a). Le pasás datos al componente.

Flujo en una sola dirección:

los datos bajan del padre al hijo. El hijo no los modifica.

Listas: .map() genera un componente por cada elemento del array

```
{topics.map((t) => <Card key={t.id} title={t.title} />) }
```

La key ayuda a React a identificar cada item.

Eventos y Estado (useState)

```
function Like() {  
  const [likes, setLikes] = useState(0)  
  
  function handleClick() {  
    setLikes(likes + 1)  
  }  
  
  return <button onClick={handleClick}>  
    ♥ {likes}  
  </button>  
}
```

Estado

información que cambia con el tiempo (los likes).

useState(0)

devuelve [valor, función para actualizarlo].

Evento onClick

al hacer clic, llama a handleClick.

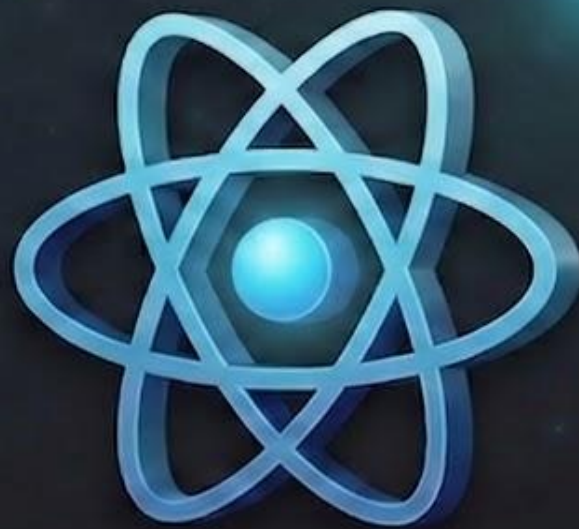
Re-render

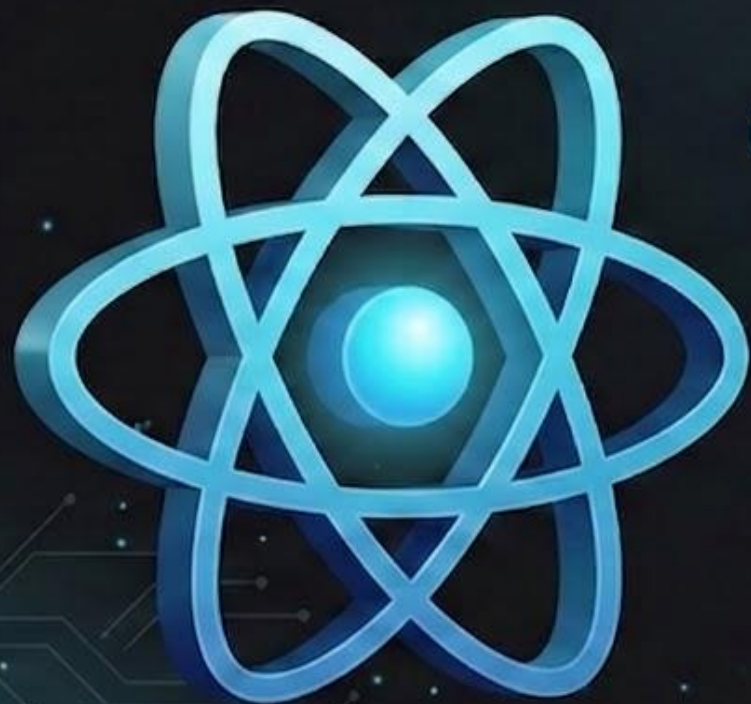
setLikes cambia el estado → React redibuja.

DEMO EN VIVO

Fundamentos de React

Una tarjeta por concepto, con botón de "me gusta". En una sola app: componentes, props, eventos y estado.





¿Preguntas?

Repo del demo: <https://github.com/JDMatu/react-recap-demo>